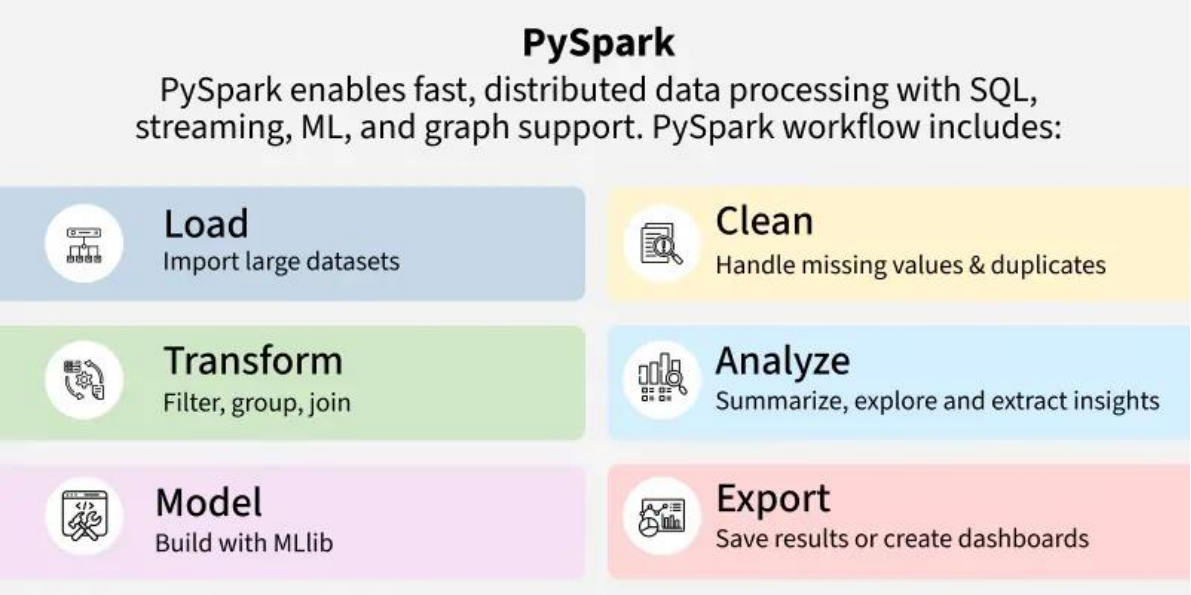
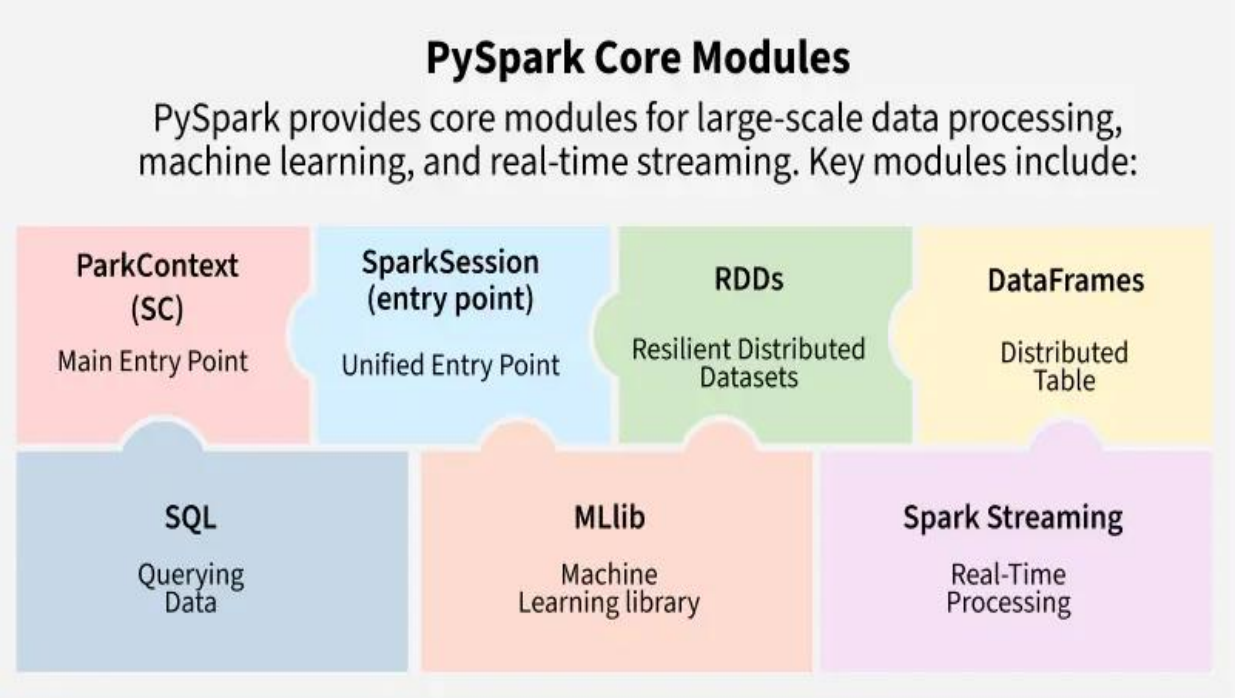
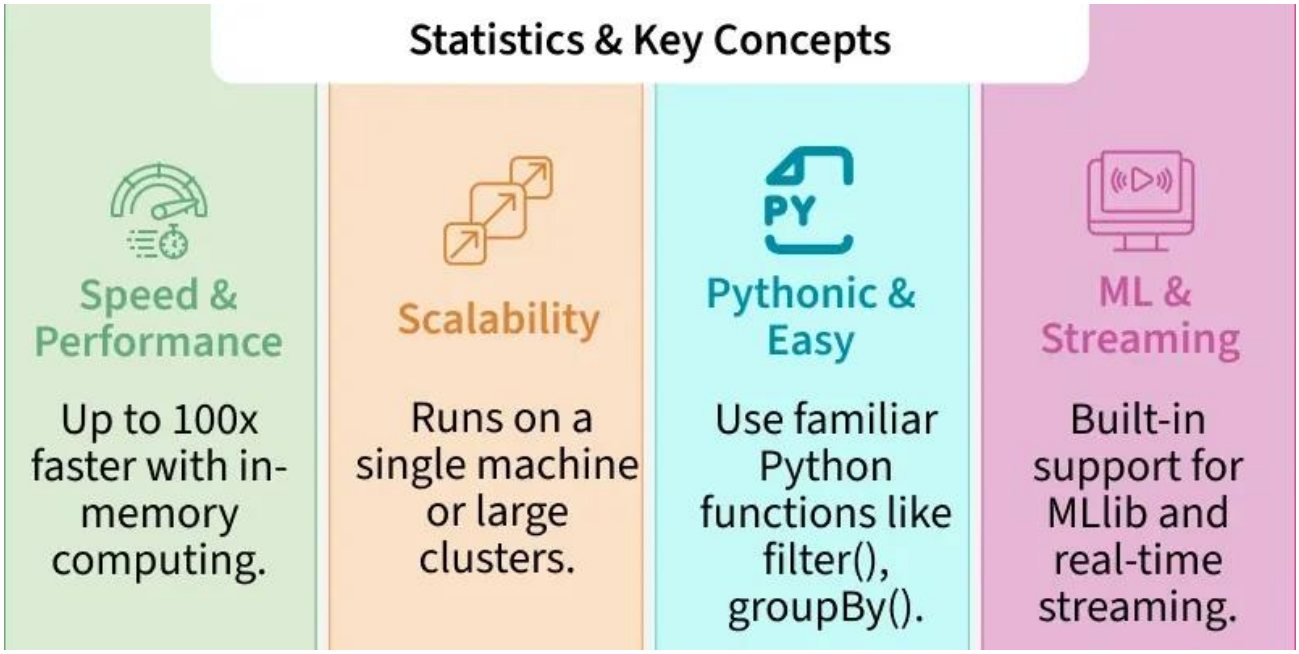
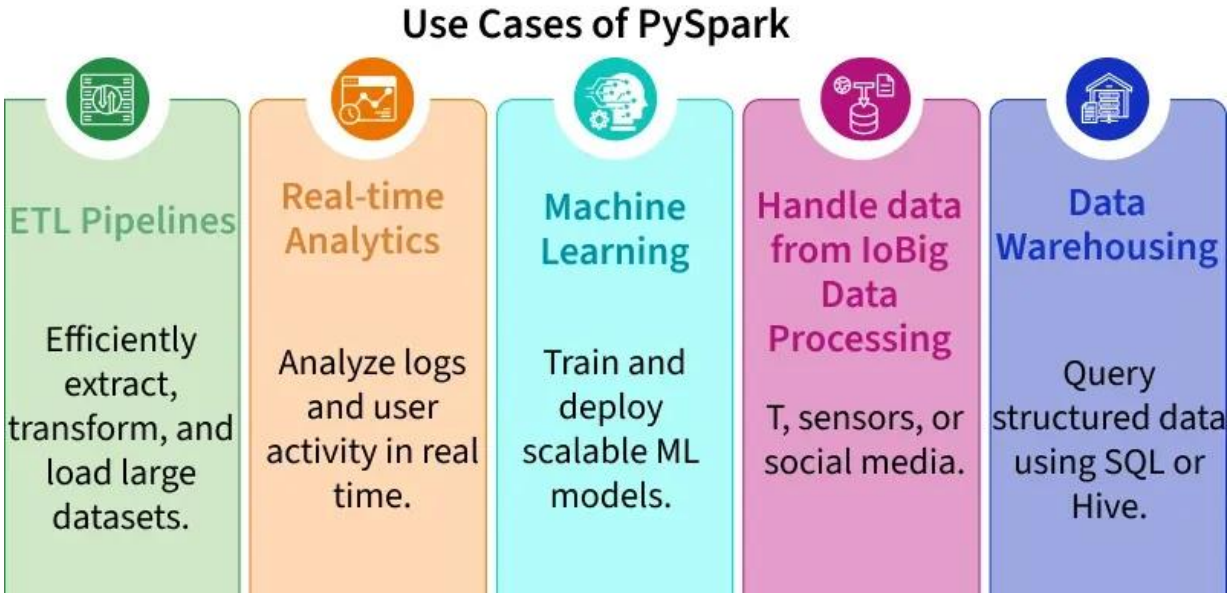
**PySpark Tutorial**

PySpark is the Python API for Apache Spark, designed for big data processing and analytics. It lets Python developers use Spark's powerful distributed computing to efficiently process large datasets across clusters. It is widely used in data analysis, machine learning and real-time processing.






***Important Facts to Know***

- ***Distributed Computing:*** *PySpark runs computations in parallel across a cluster, enabling fast data processing.*
- ***Fault Tolerance:*** *Spark recovers lost data using lineage information in resilient distributed datasets (RDDs).*
- ***Lazy Evaluation:*** *Transformations aren't executed until an action is called, allowing for optimization.*

**What is PySpark Used For?**

**PySpark** lets you use Python to process and analyze huge datasets that can't fit on one computer. It runs across many machines, making big data tasks faster and easier. You can use PySpark to:

- Perform batch and real-time processing on large datasets.
- Execute SQL queries on distributed data.
- Run scalable machine learning models.
- Stream real-time data from sources like Kafka or TCP sockets.
- Process graph data using GraphFrames.

**Why Learn PySpark?**

PySpark is one of the top tools for big data. It combines Python's simplicity with Spark's power, making it perfect for handling huge datasets.

- Enables efficient processing of petabyte-scale datasets.
- Integrates seamlessly with the Python ecosystem (pandas, NumPy, scikit-learn).
- Offers unified APIs for batch, streaming, SQL, ML and graph processing.
- Runs on Hadoop, Kubernetes, Mesos or standalone.
- Powering companies like Walmart, Trivago and many more.

**Introduction to PySpark | Distributed Computing with Apache Spark**

As data grows rapidly from sources like social media and e-commerce, traditional systems fall short. Distributed computing, with tools like Apache Spark and PySpark, enables fast, scalable data processing. This article covers the basics, key features and a hands-on PySpark.

**What is Distributed Computing?**

**Distributed computing** is a computing model where large computational tasks are divided and executed across multiple machines (nodes) that work in parallel. Think of it as breaking a huge job into smaller parts and assigning each part to a different worker. It's key features include:

- **Speed:** Multiple nodes work simultaneously.
- **Scalability:** Add more nodes to handle more data.
- **Fault tolerance:** If one node fails, others continue.

**What is Apache Spark?**

Apache Spark is an open-source distributed computing engine developed by the Apache Software Foundation. It is designed to process large datasets quickly and efficiently across a cluster of machines. It's key features include:

- **High Performance:** Much faster than Hadoop, thanks to in-memory computing.
- **Multi-language Support:** Works with Python, Scala, Java and R.
- **All-in-One Engine:** Handles batch, streaming, ML and graph processing.
- **Easy to Use:** Offers simple, high-level APIs built on the MapReduce model.

**What is Pyspark?**

**PySpark** is the Python API for Apache Spark, allowing Python developers to use the full power of Spark's distributed computing framework with familiar Python syntax. It bridges the gap between Python's ease of use and Spark's processing power. It's key features include:

- **Python-Friendly:** Build Spark applications using pure Python great for data scientists and engineers.
- **Handles Big Data:** Efficiently process huge datasets across multiple machines.
- **Rich Libraries:** Includes modules for SQL (pyspark.sql), machine learning (pyspark.ml), and streaming (pyspark.streaming).
- **DataFrame & SQL API:** Work with structured data using powerful, SQL-like operations.

**PySpark Modules**

PySpark is built in a modular way, offering specialized libraries for different data processing tasks:

| Module | Description |
|---|---|
| **pyspark.sql** | Work with structured data using DataFrames and SQL queries. |
| **pyspark.ml** | Build machine learning pipelines (classification, regression, clustering, etc.). |
| **pyspark.streaming** | Process real-time data streams (e.g., Twitter feed, logs). |
| **pyspark.graphx** | Handle graph computations and social network analysis (Scala/Java primarily). |

**How PySpark Works**

When you run a PySpark application, it follows a structured workflow to process large datasets efficiently across a distributed cluster. Here's a high-level overview:

1. **Driver Program:** Your Python script that initiates and controls the Spark job.
2. **SparkContext:** Connects the driver to the Spark cluster and manages job configuration.
3. **RDDs/DataFrames:** Data structures that are distributed and processed in parallel.

4. **Cluster Manager:** Schedules and allocates resources to worker nodes (e.g., YARN, Mesos, Kubernetes).
5. **Executor Nodes:** Run the actual tasks in parallel and return results to the driver.

**Basic Example: Word Count with PySpark**

Here's a simple PySpark example that reads a text file and counts the frequency of each word:

```
from pyspark import SparkContext

sc = SparkContext("local", "WordCount")

txt = "PySpark makes big data processing fast and easy with Python"

rdd = sc.parallelize([txt])


counts = rdd.flatMap(lambda x: x.split()) \

    .map(lambda word: (word, 1)) \

    .reduceByKey(lambda a, b: a + b)


print(counts.collect())

sc.stop()
```

**Output**

*[('PySpark', 1), ('makes', 1), ('big', 1), ('data', 1), ('processing', 1), ('fast', 1), ('and', 1), ('easy', 1), ('with', 1), ('Python', 1)]*

**Explanation:**

- **sc.parallelize()** creates an RDD from the text. flatMap() splits lines into words, map() creates (word, 1) pairs and reduceByKey() adds up word counts.
- **collect()** gathers the final word count output from all Spark worker nodes to the driver.
- **sc.stop()** stops the SparkContext to free up system resources.

**Suggested Quiz**

5 Questions

Which of the following is true about PySpark?

- **A**

PySpark is the Java API for Apache Spark

- **B**

PySpark is used for data storage

- **C**

PySpark is a version of Apache Hive

- **D**

PySpark allows Python developers to use Spark for big data processing

**Suggested Quiz**

5 Questions

Which of the following is true about PySpark?

- **A**

PySpark is the Java API for Apache Spark

- **B**

PySpark is used for data storage

- **C**

PySpark is a version of Apache Hive

- **D**

PySpark allows Python developers to use Spark for big data processing

What is the primary purpose of PySpark?

- **A**

To perform batch data processing only

- **B**

To provide a Python API for Apache Spark for big data processing

- **C**

To create machine learning models using Python

- **D**

To handle graph processing in real-time

**How to Install PySpark in Jupyter Notebook**

Last Updated : 23 Jul, 2025

- 
- 
- 

PySpark is a Python library for Apache Spark, a powerful framework for big data processing and analytics. Integrating PySpark with Jupyter Notebook provides an interactive environment for data analysis with Spark. In this article, we will know how to install PySpark in Jupyter Notebook.

**Setting Up Jupyter Notebook**

If it's not already, install Jupyter Notebook using pip:

pip install notebook

**Installing PySpark**

Install PySpark using pip:

pip install pyspark

**Example Code**

Below is a basic PySpark example in a Jupyter Notebook cell:

*# Import PySpark and initialize Spark session*

**import pyspark**

**from pyspark.sql import** SparkSession

*# Create a Spark session*

spark = SparkSession.builder.appName("PySparkExample").getOrCreate()

*# Create a DataFrame with sample data*

data = [("Alice", 25), ("Bob", 30), ("Charlie", 35)]

df = spark.createDataFrame(data, ["Name", "Age"])

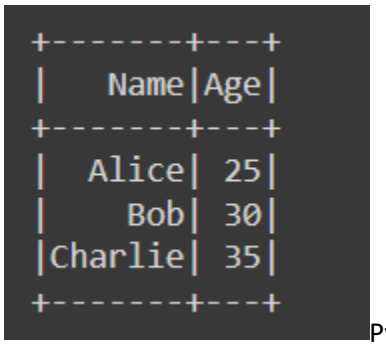
*# Show the DataFrame*

df.show()

*# Stop the Spark session*

spark.stop()

**Output**



PySpark Example

**Installation Video**

**Best Practices**

- **Configure Spark settings for optimal performance:** Adjust settings like memory allocation and parallelism based on the data and environment.
- **Use Spark's DataFrame API for efficient data manipulation:** Leverage the DataFrame API for handling large datasets efficiently.
- **Consider using Spark's MLlib for machine learning tasks:** Utilize MLlib for scalable machine learning applications.

**Q1: How do I resolve dependency conflicts?**

**Ans:** Use virtual environments to manage separate Python environments for different projects.

**Q2: Where can I find more PySpark examples?**

**Ans:** The Apache Spark documentation and various online tutorials provide extensive examples.

**How to Install PySpark in Kaggle**

Last Updated : 14 Mar, 2026

- 
- 
- 

PySpark is a Python API for distributed data processing built on Apache Spark, an open-source big data framework maintained by the Apache Software Foundation. It enables fast processing of large datasets using parallel computing. Running PySpark in Kaggle notebooks allows cloud-based big data analysis without complex local setup.

**Why PySpark in Kaggle Matters:**

- Helps practice real big data processing in a cloud-based environment
- Useful for data science competitions and large dataset analysis
- Saves time by avoiding local Spark installation and configuration
- Enables scalable data processing even on limited local hardware

***Prerequisites:***

*Before proceeding, ensure:*

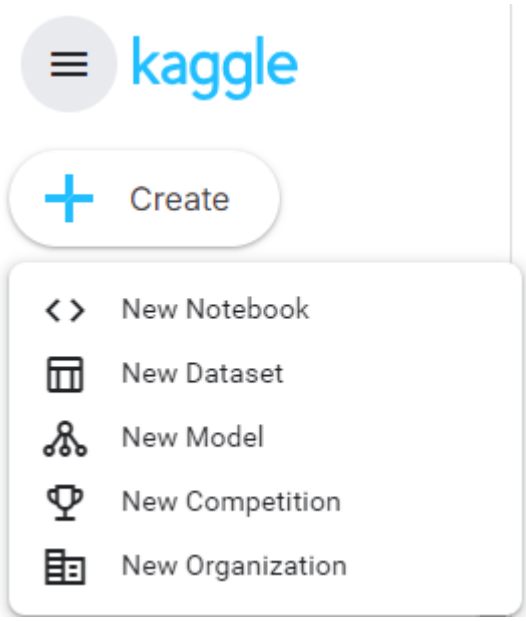
- *Active Kaggle account with notebook access*
- *Basic Python knowledge*
- *Basic understanding of distributed computing (optional but helpful)*

**How to Install PySpark via Kaggle Notebook**

To install PySpark in Kaggle, follow these simple steps:

**Step 1: Open New Kaggle Notebook**

- Sign in to your Kaggle account
- Click Create → New Notebook
- Wait for the notebook environment to start



Create new notebook

**Step 2: Install PySpark**

Now in the first cell of Kaggle Notebook type the following python code to install PySpark. Make sure you are connected to internet.

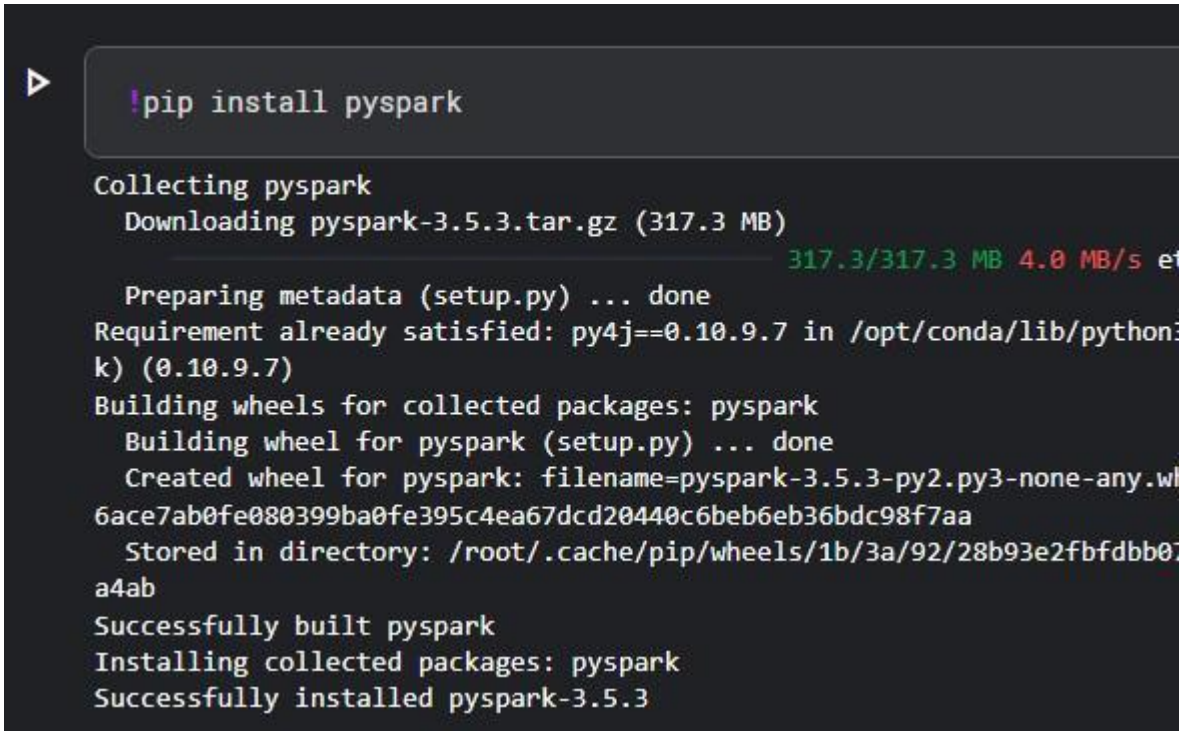
*!pip install pyspark*

Then:

- Press Shift + Enter OR
- Click Run Cell

This installs PySpark in the notebook environment.

Start the execution




Installation successful

**Step 3: Import PySpark**

- After installation, import it:

*import pyspark*

**Step 4: Initialize Spark Session**

Create a Spark session:

from pyspark.sql import SparkSession

# Initialize Spark Session

spark = SparkSession.builder \

.master("local") \

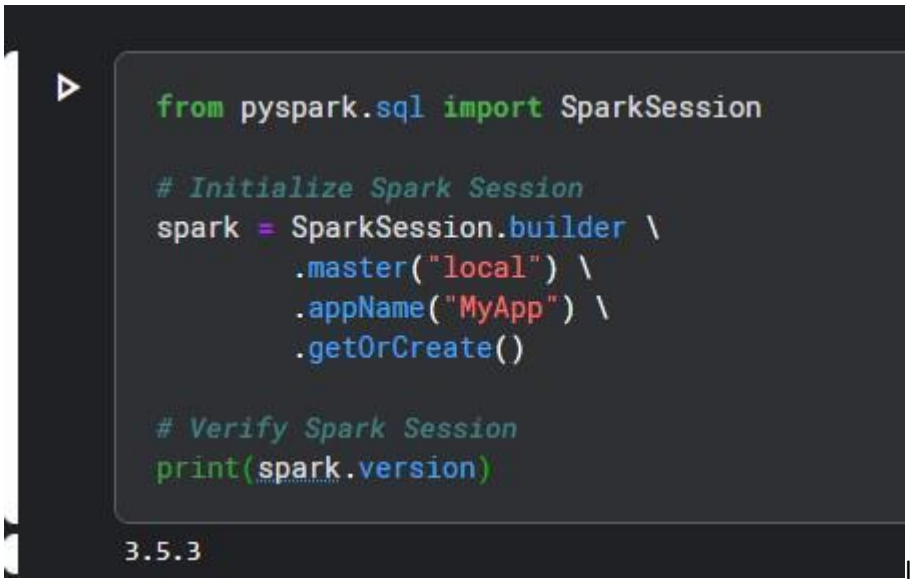
.appName("MyApp") \

.getOrCreate()

# Verify Spark Session

print(spark.version)

**Output:**
This starts a local Spark session inside Kaggle.



Initializing spark session

**Step: 5 Verify the PySpark Installation**

To ensure that PySpark is correctly installed you can verify it by running simple example.

- **Check Spark Version:** This is an easiest way to verify if the spark is installed or not. To check the version of Spark type the following code.

*pyspark.__version__*



Verifying the PySpark Installation

- **Simple PySpark Example:** Another method to verify the installation of PySpark is to run a simple example. Try this sample code to ensure everything works fine.

data = [('Rahul', 25), ('Aman', 30), ('Ravi', 28)]

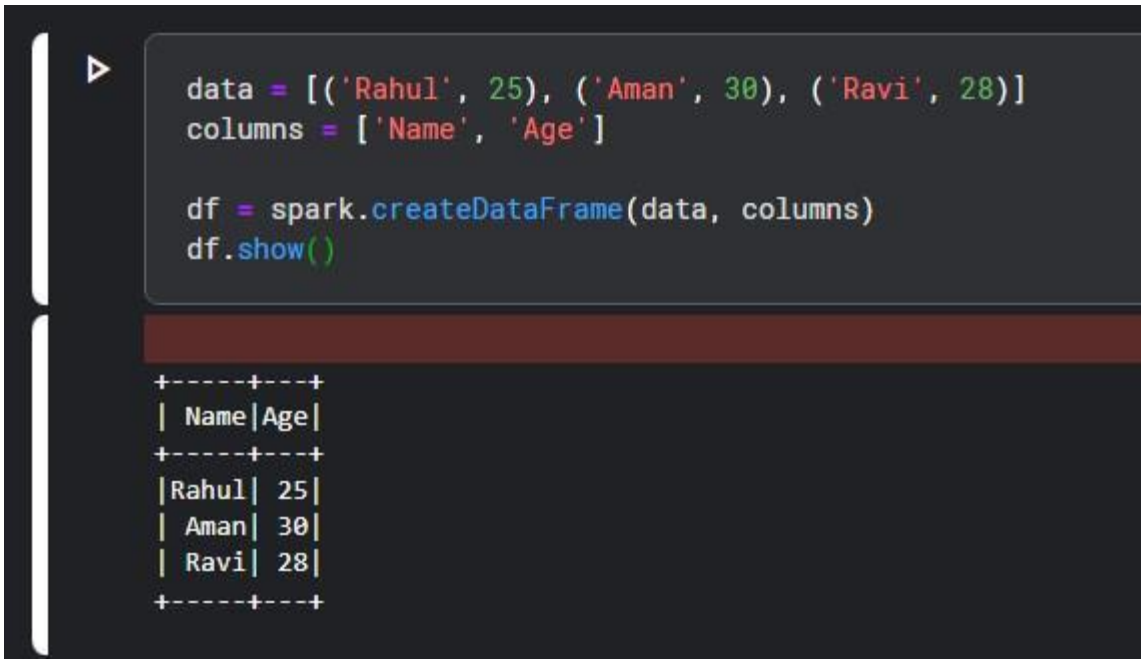
columns = ['Name', 'Age']

df = spark.createDataFrame(data, columns)

df.show()

**Output:**

- This code creates PySpark DataFrame and displays it. If the table with names and ages appears then it means PySpark is running properly and installed correctly in your Kaggle notebook.

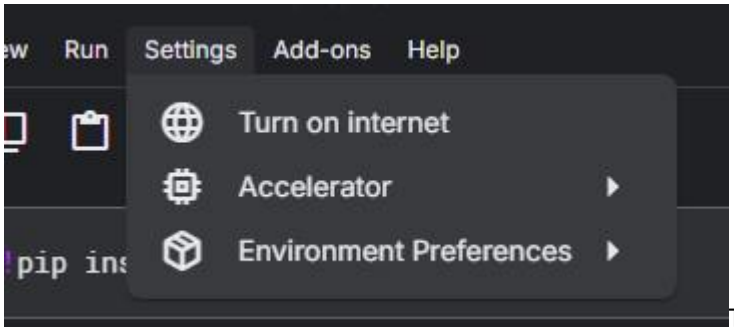


PySpark example to ensure installation

**Troubleshooting Common Issues**

Some issues can be raised while installing PySpark in Kaggle, but it can be troubleshoot and can be fixed. Some possible issues are listed below:

- **Installation Failure**: If your installation fails, then check whether your notebook has Internet and whether you have well-typed the command. Try to 'Turn on internet' in the notebook settings.



Turn on internet option always work

- **Spark session not launching**: If Spark does not start up, it can be rebooted through Runtime > Restart. Always use the correct version of PySpark.
- **Memory Limit Issues:** Kaggle notebooks have resource limits.
- **Solutions:**
    - Use smaller datasets

- Process data in batches
- Cache only required data

**How to Check PySpark Version**

Last Updated : 23 Jul, 2025

Knowing the version of PySpark you're working with is crucial for compatibility and troubleshooting purposes. In this article, we will walk through the steps to check the PySpark version in the environment.

**What is PySpark?**

PySpark is the Python API for Apache Spark, a powerful distributed computing system that allows for large-scale data processing. It enables data scientists and analysts to harness Spark's capabilities using Python, making it easier to build and deploy data pipelines and machine learning models.

**Setting Up PySpark**

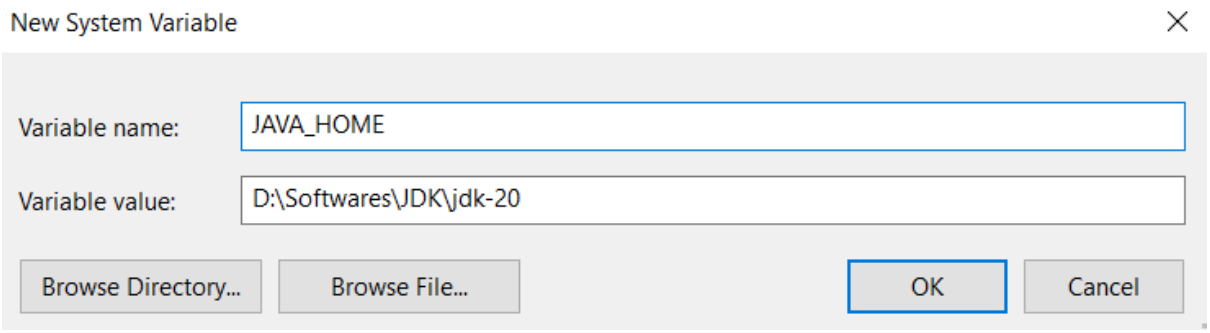
Before setting up Pyspark on the system, make sure that the latest version of Python is installed on the system. Now let us see how we can set up Pyspark on our systems.

**Install JDK**

The first step required to use Pyspark is the Java Development Kit (JDK) from this Oracle Website. Run the installer and follow the instruction on the screen to install Java on your system.

**Set JDK Environment Variable**

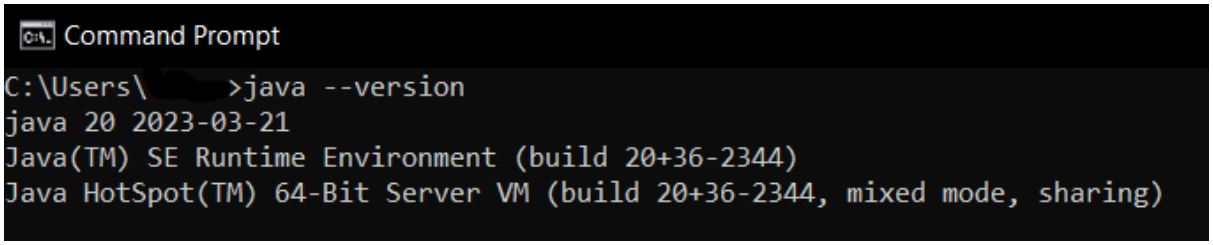
Once it is installed and set up on the system, make sure to add the JDK installation path to the system environment variables. You can do this by going to the **System Properties.** Then under the **Advanced** tab, click on **Environment variables.** Under the section **System Variables,** click on new and enter "**JAVA_HOME**" variable name and value the location where you'd JDK is installed on your system and then press OK.




Setting up JDK Environment Path

**Verify Java Installation**

Once this is done, you can check if java is properly installed on your system by typing the following command in your command prompt and press enter. This command tells the Java version installed on the system.

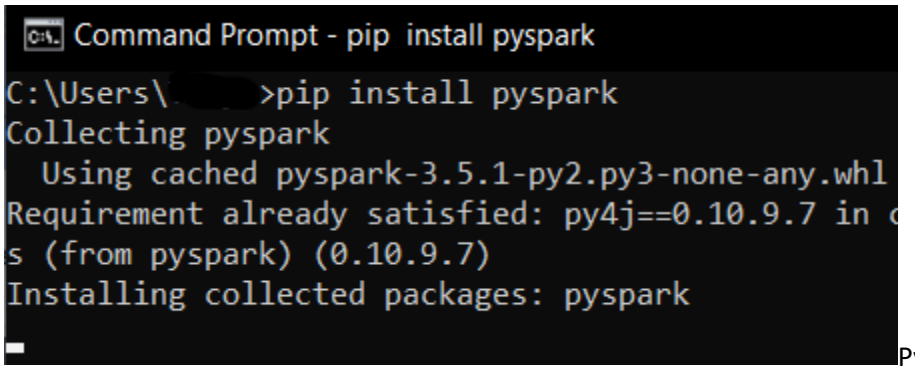



Verifying Java Installation

**Install PySpark**

Next, install PySpark on your system by opening the terminal and typing the following command in it and press enter.

pip install pyspark




PySpark Installation

**Set PySpark Environment Variable**

Similar to setting up JDK environment variable, set **"SPARK_HOME"** in environment variables for Pyspark as well.

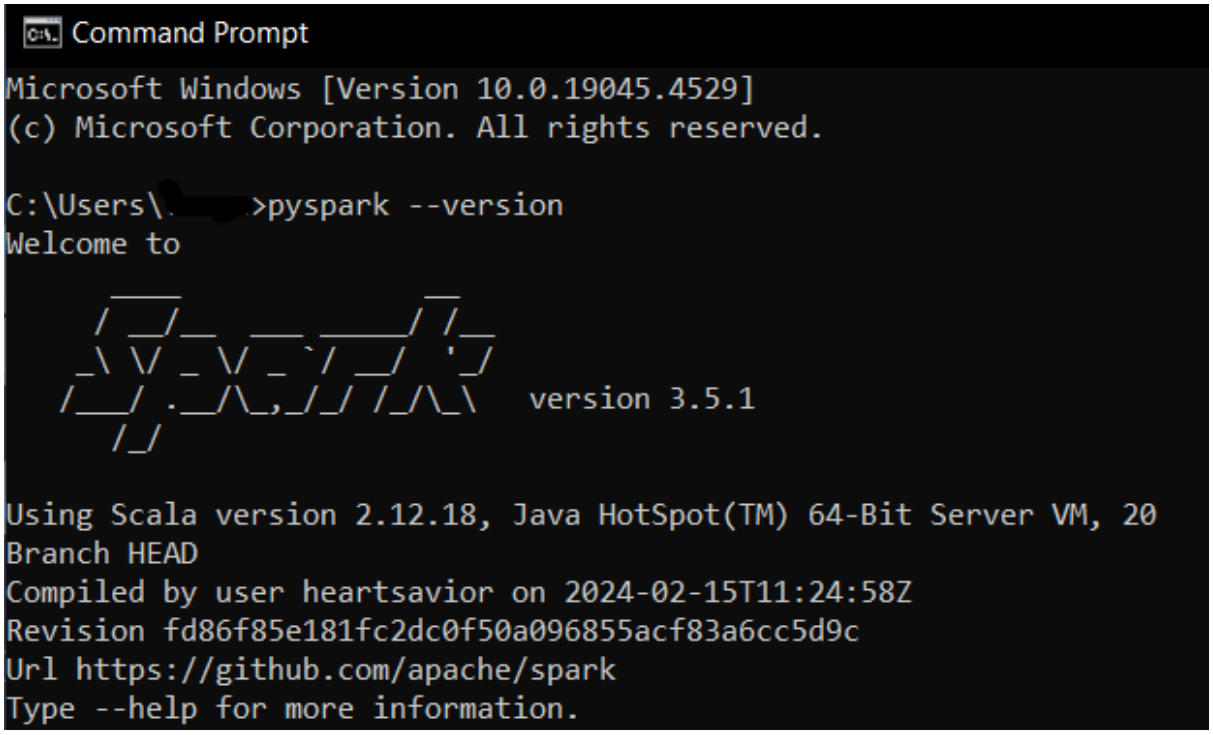
**Checking PySpark Version**

Once you are done with all the necessary installations and setting up environment variables for the system, you can now check and verify the PySpark installation and version. Let us see a few different ways to check the PySpark version.

**Using Command Prompt**

The simple and straightforward way to check ant software's version is by using the command prompt or the terminal of your system. For checking Pyspark version, you can simply type the following command in the terminal. As soon as you press enter, it will tell you the Pyspark version installed on the system.

pyspark --version



PySpark Version

**Using pyspark.version**

This method requires to tu run a basic Python script. The pyspark.__version__ method tells the current PySpark version installed on the system.

*# import pyspark module*

**import pyspark**

*# Get the version of PySpark*

pyspark_version = pyspark.__version__

print(f"PySpark Version: **{**pyspark_version**}**")

**Output:**

PySpark Version: 3.5.1

**Working with PySpark**

Start working with data using RDDs and DataFrames for distributed processing.

**Creating RDDs and DataFrames:** Build DataFrames in multiple ways and define custom schemas for better control.

**PySpark - Read CSV file into DataFrame**

Last Updated : 25 Oct, 2021

- 
-

In this article, we are going to see how to read CSV files into Dataframe. For this, we will use Pyspark and Python.

**Files Used:**

- authors
- book_author
- books

**Read CSV File into DataFrame**

Here we are going to read a single CSV into dataframe using spark.read.csv and then create dataframe with this data using .toPandas().

```
from pyspark.sql import SparkSession


spark = SparkSession.builder.appName(
    'Read CSV File into DataFrame').getOrCreate()


authors = spark.read.csv('/content/authors.csv', sep=',',
                         inferSchema=True, header=True)


df = authors.toPandas()

df.head()
```

**Output:**

| | AUTHOR_ID | AUTHOR_NAME |
|---|---|---|
| 0 | 1 | A. Kannan |
| 1 | 2 | Aaron M. Tenenbaum |
| 2 | 3 | Adam Smith |
| 3 | 4 | Adity Gupta |
| 4 | 5 | Alex Ionescu |
| ... | ... | ... |
| 326 | 327 | Yashavant P. Kanetkar |
| 327 | 328 | Yedidyah Langram |
| 328 | 329 | Yochay Kiriaty |
| 329 | 330 | Yukihisa Kageyama |
| 330 | 331 | David Patterson |

331 rows × 2 columns

Here, we passed our CSV file authors.csv. Second, we passed the delimiter used in the CSV file. Here the delimiter is comma **','**. Next, we set the **inferSchema** attribute as **True**, this will go through the CSV file and automatically adapt its schema into PySpark Dataframe. Then, we converted the PySpark Dataframe to Pandas Dataframe **df** using **toPandas()** method.

**Read Multiple CSV Files**

To read multiple CSV files, we will pass a python list of paths of the CSV files as **string** type.

```
from pyspark.sql import SparkSession


spark = SparkSession.builder.appName('Read Multiple CSV Files').getOrCreate()


path = ['/content/authors.csv',
        '/content/book_author.csv']


files = spark.read.csv(path, sep=',',
                       inferSchema=True, header=True)


df1 = files.toPandas()
display(df1.head())
display(df1.tail())
```

**Output:**

| | AUTHOR_ID | AUTHOR_NAME |
|---|---|---|
| **0** | 1 | A. Kannan |
| **1** | 2 | Aaron M. Tenenbaum |
| **2** | 3 | Adam Smith |
| **3** | 4 | Adity Gupta |
| **4** | 5 | Alex Ionescu |

| | AUTHOR_ID | AUTHOR_NAME |
|---|---|---|
| **724** | B0209 | 153 |
| **725** | B0210 | 292 |
| **726** | B0211 | 317 |
| **727** | B0212 | 278 |
| **728** | B0213 | 287 |

Here, we imported authors.csv and book_author.csv present in the same current working directory having delimiter as comma '**,**' and the first row as Header.

**Read All CSV Files in Directory**

To read all CSV files in the directory, we will use ***** for considering each file in the directory.

**from pyspark.sql import** SparkSession

```
spark = SparkSession.builder.appName(
    'Read All CSV Files in Directory').getOrCreate()

file2 = spark.read.csv('/content/*.csv', sep=',',
                       inferSchema=True, header=True)

df1 = file2.toPandas()
display(df1.head())
display(df1.tail())
```

**Output:**

| | AUTHOR_ID | AUTHOR_NAME |
|---|---|---|
| **0** | 1 | A. Kannan |
| **1** | 2 | Aaron M. Tenenbaum |
| **2** | 3 | Adam Smith |
| **3** | 4 | Adity Gupta |
| **4** | 5 | Alex lonescu |

| | AUTHOR_ID | AUTHOR_NAME |
|---|---|---|
| **799** | Ind | 97711 |
| **800** | Ind | 3956 |
| **801** | Aus | 519952 |
| **802** | Aus | 16116 |
| **803** | NZ | 609483 |

This will read all the **CSV** files present in the current working directory, having delimiter as comma '**,**' and the first row as Header.